



TrustClaw Email Agent

Forward Deployed AI Architect — JFrog Evaluation Assignment

Victor Ramirez · June 2026

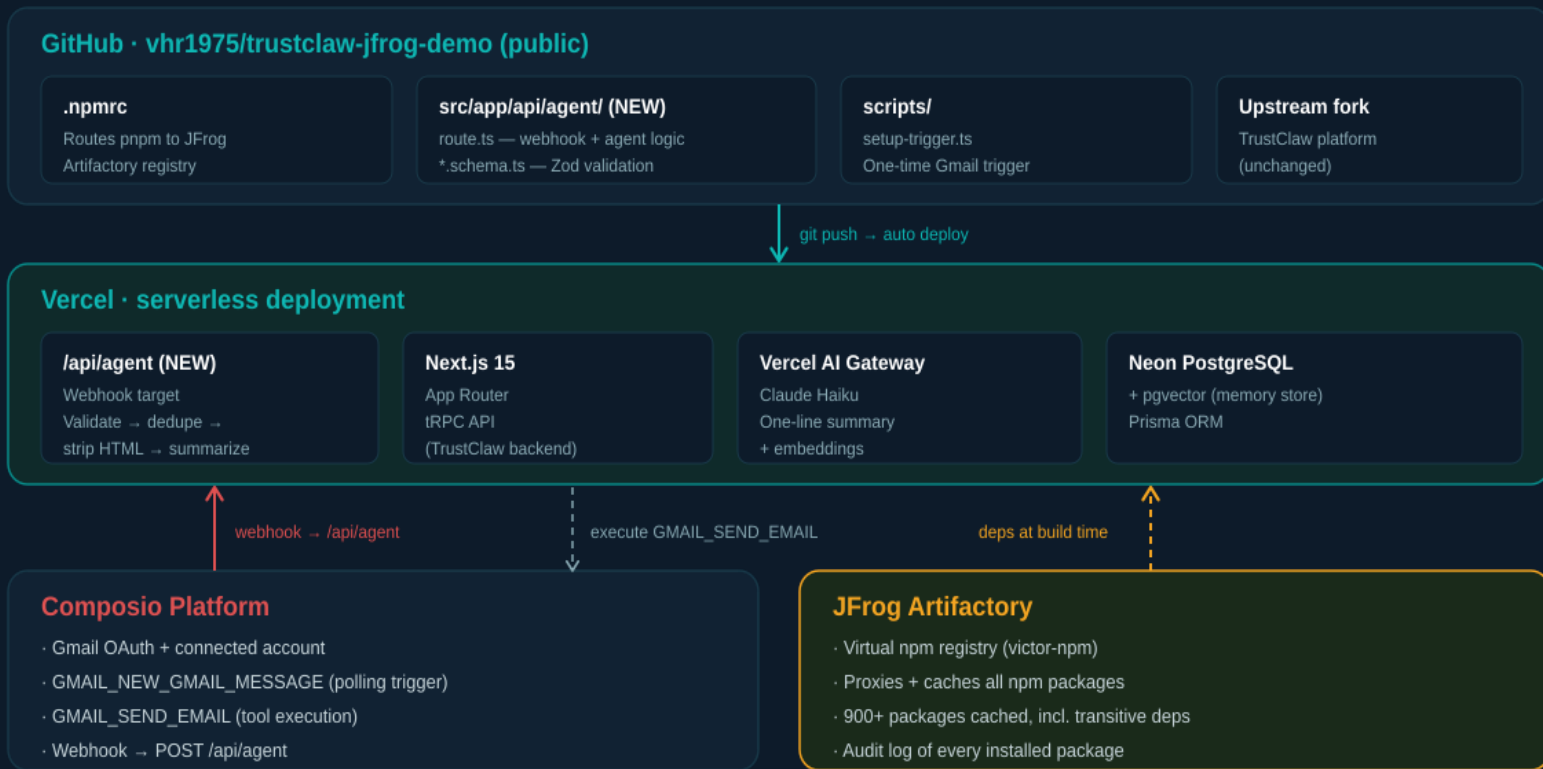
GitHub: <https://github.com/vhr1975/trustclaw-jfrog-demo>

Vercel: <https://trustclaw-jfrog-demo.vercel.app/>

An autonomous email agent — reads incoming Gmail, summarizes in one line via Claude, replies with AGENT SUMMARY {summary}. All npm dependencies routed through JFrog Artifactory. No human in the loop.

System Architecture

Component map — GitHub deploys to Vercel; Composio and JFrog are parallel dependencies



How It Works

Runtime agent flow + Build-time dependency supply chain

RUNTIME — EMAIL AGENT FLOW



Incoming email

Gmail inbox — Composio polling trigger



Composio webhook

POST /api/agent with payload



Vercel serverless fn

Validate → dedupe → strip HTML → summarize



Claude Haiku via AI GW

One-line summary generated



GMAIL_SEND_EMAIL

Reply: AGENT SUMMARY {summary} + original body

BUILD-TIME — DEPENDENCY SUPPLY CHAIN



Developer pnpm install



JFrog Artifactory

victor-npm virtual repo
900+ packages cached



npmjs.org public registry

Cache miss → fetch upstream



Vercel production build

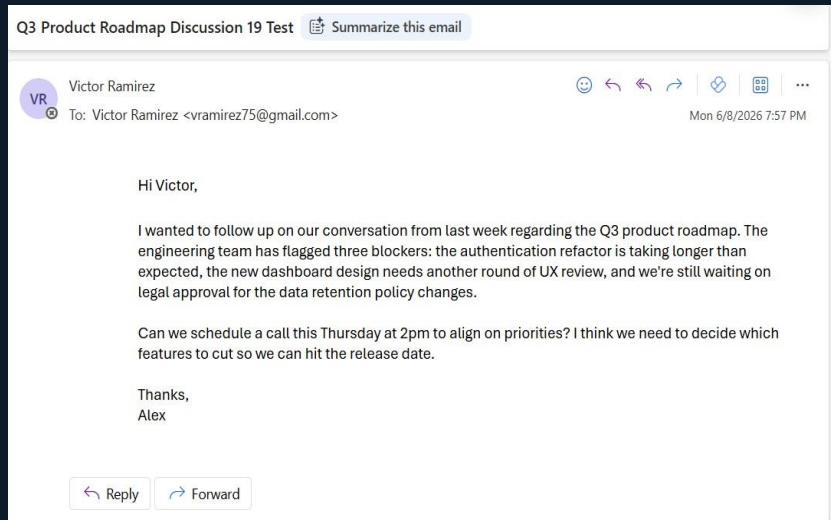
Verified, cached packages only

Every package — including transitive deps — gets an immutable audit trail, vulnerability scan, and supply chain control.

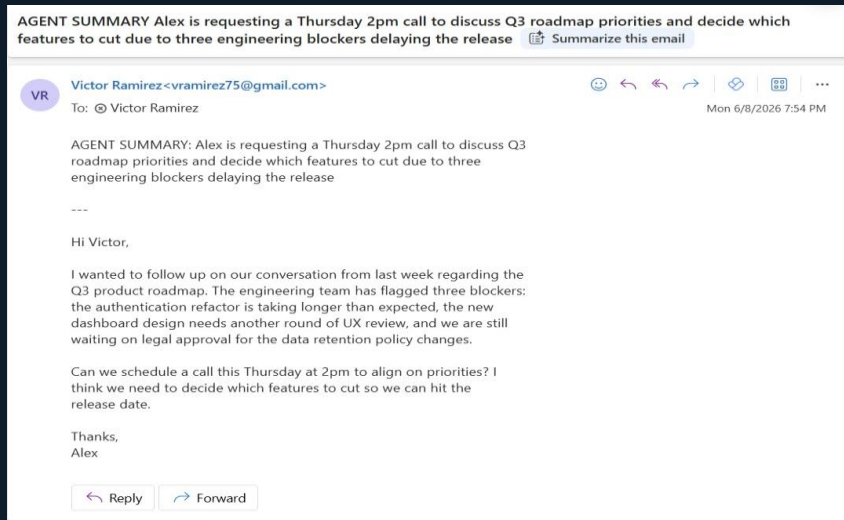
It Works

End-to-end proof — email in, AGENT SUMMARY out

INPUT — Email sent



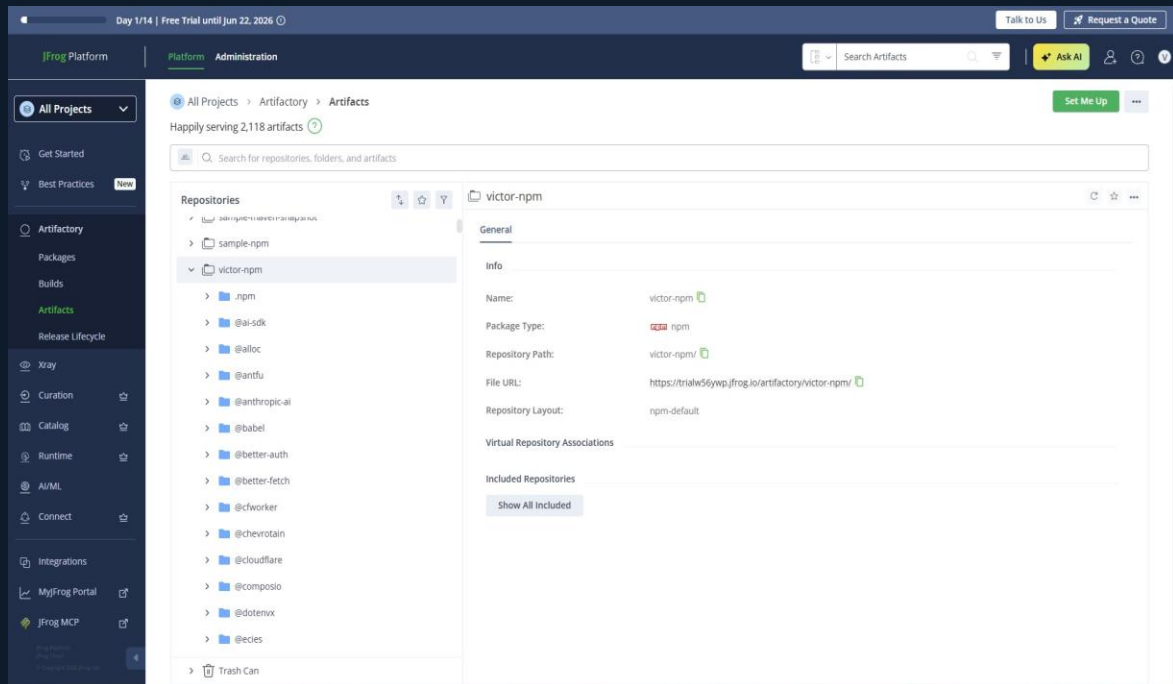
OUTPUT — Agent reply received



Subject generated: "AGENT SUMMARY Alex is requesting a Thursday 2pm call to discuss Q3 roadmap priorities and decide which features to cut due to three engineering blockers delaying the release"

JFrog Artifactory — In Use

All npm dependencies routed through victor-npm virtual repository: <https://trialw56ywp.jfrog.io/ui/repos/tree/General/victor-npm>



900+

Packages
Cached

victor-npm

Virtual
Repository

.npmrc

Registry
Config

Why this matters: Every package — including transitive dependencies — is routed through JFrog first. The team gets an immutable audit trail, vulnerability scanning, and the ability to block malicious packages before they reach production.

Where Developer Experience Breaks

Five friction points documented during implementation

#1



GMAIL_REPLY_TO_THREAD silently ignores subject parameter

HIGH

Code returned successful: true. Core requirement was silently not met. Silent failures are the hardest bugs to find.

#2



Webhook payload schema not documented for v3

HIGH

Payload shape differed from all docs examples. Discovery required inspecting live webhook logs — one JSON example eliminates this for every developer.

#3



TOOL_VERSION_REQUIRED with no resolution path

MED

Fix (dangerouslySkipVersionCheck: true) only exists resolved after log review. A developer hitting this at 11pm files a support ticket or quits.

#4



ConnectedAccountEntityIdMismatch — userId coupling opaque

MED

The relationship between user IDs and connected account IDs is not explained in getting-started docs. Cryptic mismatch error with no actionable message.

#5



Polling interval unpredictable (15 to 60+ min)

LOW

GMAIL_NEW_GMAIL_MESSAGE described as ~15 min but varies widely. Production implementations need Gmail Pub/Sub push notifications for reliability.

Recommended Fixes

Concrete, low-effort improvements — high developer impact

Problem: Silent subject failure

EFFORT: Low

Fix: When a parameter is unsupported, validate and reject at the API boundary rather than silently no-op. (My fix: switched to GMAIL_SEND_EMAIL to control the subject directly.)

Problem: Undocumented webhook schema

EFFORT: Low

Fix: Add one JSON payload example to the email trigger docs. Eliminates the most common first-30-minutes debugging session.

Problem: Dead-end version error

EFFORT: Low

Fix: Surface dangerouslySkipVersionCheck in the error message itself. Move the fix from a log output.

Problem: Opaque userId/account coupling

EFFORT: Med

Fix: Explain the user entity / connected account relationship in getting-started docs. Add a validation message that names the mismatch.

Problem: Unpredictable polling

EFFORT: Med

Fix: Document Gmail Pub/Sub push notifications as the production path. Position polling as a quickstart-only option.

What This Means in the Field

This exercise is a microcosm of what forward-deployed looks like



Silent failures at scale

A customer building AI workflows on Composio + JFrog hits this and assumes JFrog is the problem. Forward-deployed means you find it before they do.



Docs gaps = adoption gaps

Every developer who quits at the webhook setup step never reaches the JFrog integration. DX quality directly controls ecosystem reach.



Production reality vs. demos

This agent worked in testing. It silently broke in production. That gap — POC to prod — is exactly what JFrog exists to close.



Governance at every layer

AI-generated code, third-party agents, npm deps — all need the same visibility and policy enforcement. All the way to the binary.



Built. Deployed. Documented.

Ready to bring this thinking to JFrog customers.

GitHub: <https://github.com/vhr1975/trustclaw-jfrog-demo>

Vercel: <https://trustclaw-jfrog-demo.vercel.app/>

Victor Ramirez · VRamirez75@gmail.com · (415) 299-4692